Trello Clone - Technical Documentation

Project: Trello Clone Task Management Application
Version: 1.0.0 (Phase 1 Complete)
Last Updated: April 2026
Team: Pratham, Prisha Chaudhary

Table of Contents

1. Project Overview
2. Architecture
3. Technology Stack
4. Features & Functionality
5. Data Models
6. API Endpoints
7. User Flow & Workflows
8. Security Considerations
9. File Structure
10. Deployment
11. Future Enhancements

Project Overview

Purpose

A lightweight, web-based task management application inspired by Trello. It enables users to:

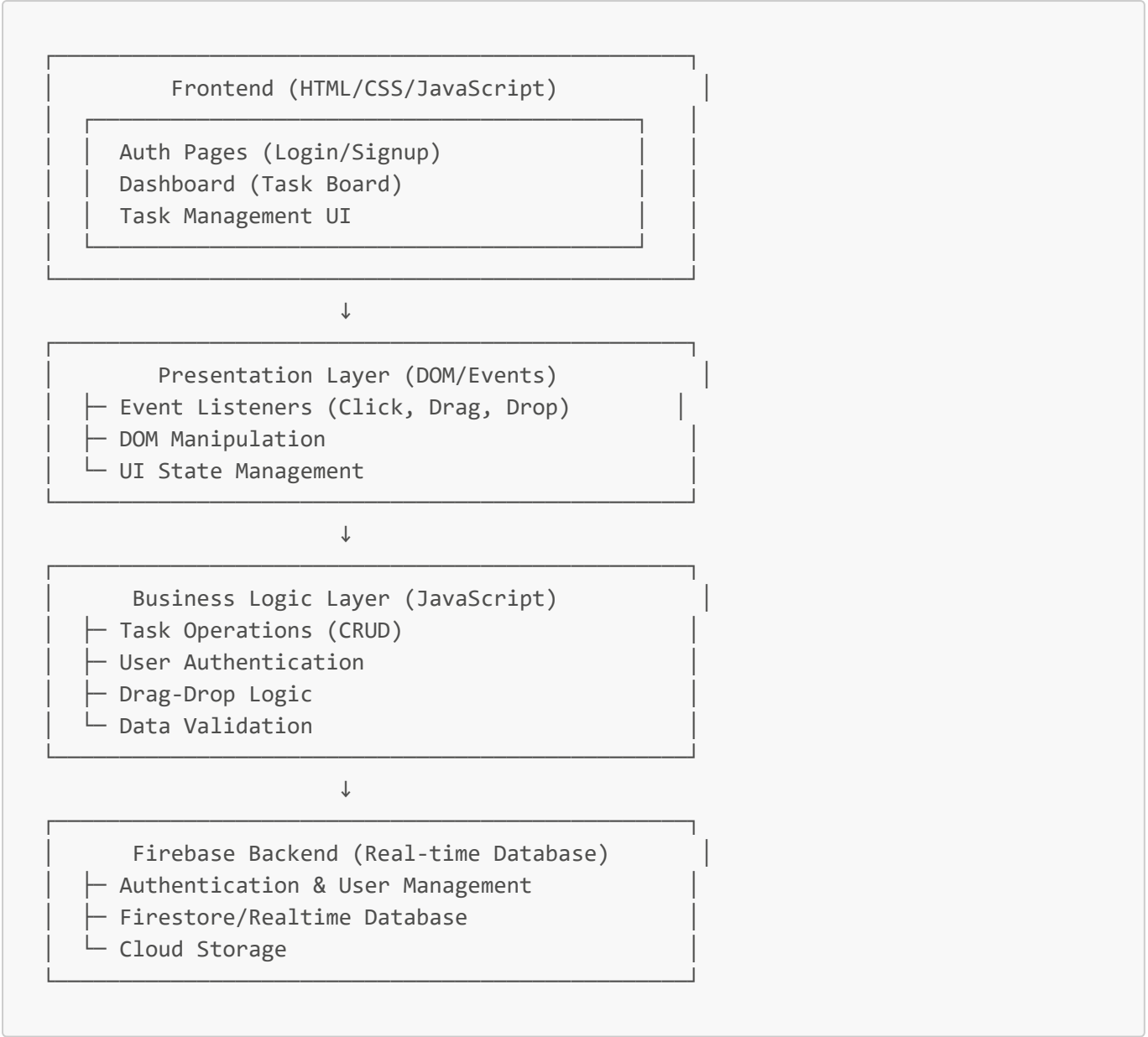
- Create and manage personal task boards
- Organize tasks by status (Pending, Running, Completed)
- Collaborate with task assignment features
- Persist data using Firebase

Key Characteristics

- **Client-side focused** with Firebase backend
- **Real-time data synchronization**
- **User authentication** with secure session management
- **Drag-and-drop interface** for intuitive task organization
- **Responsive design** for desktop and mobile

Architecture

High-Level Architecture



Component Architecture

- **Authentication Module** - Handles signup, login, logout, session management
- **Task Management Module** - CRUD operations, status tracking
- **UI Module** - Rendering, drag-drop, user interactions
- **Data Persistence Module** - Firebase integration
- **Validation Module** - Input validation, business logic enforcement

---

Technology Stack

Frontend

Technology	Version	Purpose
HTML5	Latest	Page structure & semantic markup
CSS3	Latest	Styling, responsive design, animations
JavaScript (Vanilla)	ES6+	Client-side logic, DOM manipulation

Technology	Version	Purpose
Local Storage API	Native	Client-side caching, session persistence

Backend

Technology	Purpose
Firebase Authentication	User sign-up, login, session management
Firebase Realtime Database / Firestore	Data persistence for tasks and user data
Firebase Hosting	Application deployment & hosting

Development Tools

- Git (Version Control)
- Firebase CLI
- Browser DevTools

Features & Functionality

Phase 1 - Core Features (COMPLETED)

1. **User Authentication**

- ✅ Sign up with email & password
- ✅ Login with credentials validation
- ✅ Logout functionality
- ✅ Session persistence
- ✅ Password validation rules

2. **Task Management**

- ✅ Create tasks with title and description
- ✅ View all user tasks
- ✅ Update task details
- ✅ Delete tasks
- ✅ Mark tasks with status (Pending, Running, Completed)

3. **Task Organization**

- ✅ Drag-and-drop between status columns
- ✅ Visual task cards with status indicators
- ✅ Task filtering by status
- ✅ Assign tasks to team members

4. **UI/UX Features**

- ✅ Responsive design (Desktop & Mobile)
- ✅ Clean, intuitive interface
- ✅ Real-time task updates
- ✅ Visual feedback for user actions
- ✅ Button interactions & navigation

Data Models

User Model

```
{
  userId: "unique_firebase_id",
  name: "User Full Name",
  email: "user@example.com",
  passwordHash: "hashed_password",
  createdAt: "2026-04-14T10:30:00Z",
  updatedAt: "2026-04-14T10:30:00Z",
  preferences: {
    theme: "light",
    language: "en"
  }
}
```

Task Model

```
{
  taskId: "unique_task_id",
  userId: "task_owner_id",
  title: "Task Title",
  description: "Detailed task description",
  status: "pending" | "running" | "completed",
  priority: "low" | "medium" | "high",
  assignedTo: "team_member_id",
  dueDate: "2026-04-20T23:59:59Z",
  createdAt: "2026-04-14T10:30:00Z",
  updatedAt: "2026-04-14T10:30:00Z",
  tags: ["tag1", "tag2"],
  attachments: []
}
```

Firestore Collections Structure

```
users/
├── {userId}/
│   ├── name: string
│   ├── email: string
```

```
│   └── createdAt: timestamp

tasks/
├── {taskId}/
│   ├── userId: string
│   ├── title: string
│   ├── description: string
│   ├── status: string
│   ├── priority: string
│   ├── assignedTo: string
│   ├── dueDate: timestamp
│   ├── createdAt: timestamp
│   └── updatedAt: timestamp
```

API Endpoints

Authentication Endpoints

Sign Up

```
POST /signup
Content-Type: application/json

{
  "name": "User Name",
  "email": "user@example.com",
  "password": "SecurePassword123"
}

Response: { userId, token, message }
```

Login

```
POST /login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "SecurePassword123"
}

Response: { userId, token, message }
```

Logout

```
POST /logout

Response: { message: "Logged out successfully" }
```

Task Endpoints

Create Task

```
POST /tasks
Content-Type: application/json

{
  "title": "Task Title",
  "description": "Task Description",
  "priority": "medium",
  "dueDate": "2026-04-20"
}

Response: { taskId, task }
```

Get All Tasks

```
GET /tasks

Response: { tasks: [Task] }
```

Get Task by ID

```
GET /tasks/{taskId}

Response: { task: Task }
```

Update Task

```
PUT /tasks/{taskId}
Content-Type: application/json

{
  "title": "Updated Title",
  "status": "running",
  "assignedTo": "userId"
}
```

```
Response: { task: UpdatedTask }
```

Delete Task

```
DELETE /tasks/{taskId}

Response: { message: "Task deleted successfully" }
```

Update Task Status (Drag & Drop)

```
PUT /tasks/{taskId}/status
Content-Type: application/json

{
  "status": "completed"
}

Response: { task: UpdatedTask }
```

User Flow & Workflows

Authentication Flow

```
1. User visits application
   ↓
2. Checks if logged in (localStorage/sessionStorage)
   ├─ Yes → Redirect to Dashboard
   └─ No → Show Login/Signup Page
   ↓
3. User selects Login or Signup
   ├─ Sign Up:
   │  ├─ Enter name, email, password
   │  ├─ Validate input
   │  ├─ Check if user exists
   │  ├─ Create new user in Firebase
   │  └─ Set session token
   │
   └─ Login:
      ├─ Enter email, password
      ├─ Validate credentials
      ├─ Authenticate with Firebase
      └─ Set session token
```

```
   ↓
4. Redirect to Task Dashboard
```

Task Management Workflow

```
Dashboard Load:
1. Fetch user's tasks from Firebase
2. Organize tasks by status (3 columns)
3. Display tasks as draggable cards
4. Load event listeners

User Actions:
├─ Create Task:
│  ├─ Click "Add Task" button
│  ├─ Fill form (title, description, priority)
│  ├─ Submit
│  └─ Task appears in "Pending" column
│
├─ Update Task:
│  ├─ Click on task card
│  ├─ Edit details in modal
│  ├─ Save changes
│  └─ Update reflected in Firebase
│
├─ Drag & Drop:
│  ├─ Click & hold task
│  ├─ Drag to another status column
│  ├─ Drop to update status
│  └─ Firebase updates automatically
│
├─ Assign Task:
│  ├─ Click task options menu
│  ├─ Select team member
│  ├─ Task assigned
│  └─ Assignee notified (future phase)
│
└─ Delete Task:
   ├─ Click delete button
   ├─ Confirm action
   ├─ Task removed from UI
   └─ Firebase record deleted
```

Security Considerations

Phase 1 Implementation

- ✅ Email/Password authentication via Firebase
- ✅ User session management with tokens
- ✅ Client-side input validation

- ✅ XSS protection via DOM API safety
- ✅ CSRF tokens on state-changing operations

Planned Security Enhancements (Phase 2+)

- Password strength requirements
- Password hashing with bcrypt
- Email verification
- Two-factor authentication (2FA)
- Rate limiting on login attempts
- Refresh token rotation
- HTTPS enforcement
- Data encryption at rest
- Role-based access control (RBAC)
- Audit logging for security events

File Structure

Current File Organization

```
trello-clone/
│
├── public/                    # Firebase hosting directory
│   ├── index.html            # Login/Signup page
│   ├── dashboard.html        # Task management dashboard
│   ├── styles/
│   │   ├── auth.css          # Authentication page styles
│   │   └── dashboard.css     # Dashboard styles
│   └── scripts/
│       ├── auth.js           # Authentication logic
│       ├── dashboard.js      # Task management logic
│       └── firebase-config.js # Firebase configuration
│
├── firebase.json             # Firebase hosting configuration
├── .firebaserc               # Firebase project configuration
├── .gitignore               # Git ignore rules
├── Readme.md                # Project overview
└── TECHNICAL_DOCUMENTATION.md (this file)
```

Key Files Description

File	Purpose
`index.html`	Login & Sign Up interface
`dashboard.html`	Main task board interface
`auth.js`	User authentication logic

File	Purpose
`dashboard.js`	Task operations & drag-drop
`firebase-config.js`	Firebase initialization & configuration
`auth.css`	Authentication page styling
`dashboard.css`	Dashboard & task card styling
`firebase.json`	Firebase hosting rules & config

Deployment

Current Deployment

- **Platform:** Firebase Hosting
- **Live URL:** [Your Firebase URL]
- **Environment:** Production

Deployment Steps

1. Build and test locally
2. Run `firebase login`
3. Run `firebase deploy --only hosting`
4. Verify deployment at Firebase console

Environment Configuration

```
// firebase-config.js
const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "your-project.firebaseapp.com",
  projectId: "your-project-id",
  storageBucket: "your-project.appspot.com",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
  appId: "YOUR_APP_ID"
};
```

Future Enhancements

Phase 2 - Advanced Features

- [ ] Multi-board support per user
- [ ] Board sharing & collaboration
- [ ] Real-time collaboration (multiple users on same board)
- [ ] Comments & activity logs on tasks
- [ ] File attachments to tasks

- [ ] Task templates & recurring tasks
- [ ] Advanced search & filtering
- [ ] Custom labels & categories

Phase 3 - Optimization & Scaling

- [ ] Database indexing for performance
- [ ] Caching strategies (IndexedDB, Service Workers)
- [ ] Offline mode support
- [ ] Progressive Web App (PWA) features
- [ ] Mobile app (React Native / Flutter)

Phase 4 - Enterprise Features

- [ ] Team management & roles
- [ ] Permissions & access control
- [ ] Activity audit logs
- [ ] API for third-party integrations
- [ ] Webhook support
- [ ] Analytics & reporting dashboard
- [ ] SSO integration (OAuth, SAML)

Phase 5 - Infrastructure

- [ ] Migrate to cloud database (PostgreSQL)
- [ ] Implement backend API (Node.js/Express)
- [ ] CI/CD pipeline setup
- [ ] Automated testing (Unit, Integration, E2E)
- [ ] Performance monitoring & alerting
- [ ] Load balancing & scalability

Development Guidelines

Code Standards

- Use semantic HTML5
- Follow CSS naming conventions (BEM or similar)
- Use arrow functions and modern JavaScript (ES6+)
- Add comments for complex logic
- Keep functions small and focused

Branching Strategy

- `main` - Production ready code
- `develop` - Development integration branch
- `feature/*` - Feature branches
- `bugfix/*` - Bug fix branches
- `hotfix/*` - Production hotfixes

Commit Message Format

```
[TYPE] Brief description

Detailed explanation (optional)

Related to: #issue-number
```

Type: `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`

Troubleshooting

Common Issues

Issue: User data not persisting after logout

- **Solution:** Check Firebase Firestore rules and ensure user data is being saved properly

Issue: Drag-drop not working on mobile

- **Solution:** Implement touch events in addition to mouse events

Issue: Slow task loading for users with many tasks

- **Solution:** Implement pagination or virtual scrolling in future phase

---

References & Resources

- Firebase Documentation
- MDN Web Docs
- HTML5 Drag and Drop API
- Web Security Best Practices

Document Version: 1.0
Last Updated: April 14, 2026
Maintainers: Pratham, Prisha Chaudhary